

Chapitre 3 - Caractéristiques des formats et structure des données

I.	Format des données	1
II.	Formats d'échange de données.....	2
1.	Le format Texte brut (CSV).....	2
2.	Le XML.....	2
3.	Le JSON.....	2
III.	Aspects sécuritaires sur les formats d'échange.....	2
1.	Texte brut.....	3
2.	JSON et XML.....	3

I. Format des données

Le format est primordial : Il définit ce qui peut être stocké, et les opérations supportées par le SQL.

Exemple avec un code postal : 38170.

- S'il est conservé sous forme numérique, il sera très difficile d'extraire le département (1^{er} et 2^{ème} caractères).
- Sous forme de texte, cela ne posera pas de problème, mais aucune opération de type numérique (addition, soustraction, ...) ne sera possible.

Selon le SGBD, les formats de données supportés diffèrent :

Pour MySQL :

Nom du type	Taille (octets)	Contenu
TINYINT	1	-128 à +127 ou bien 0 à 255
[UNSIGNED] SMALLINT	2	-32 768 à 32 767 ou bien 0 à 65 535
[UNSIGNED] MEDIUMINT	3	-8 388 608 à 8 388 607 ou bien 0 à 16 777 215
[UNSIGNED] INT	4	-2 147 483 648 à 2 147 483 647 ou bien 0 à 4 294 967 295
[UNSIGNED] BIGINT	8	-9 223 372 036 854 775 808 à 9 223 372 036 854 775 807 ou bien 0 à 18 446 744 073 709 551 615
FLOAT	4	-1,175494351 E-38 à 3,402823466 E+38
DOUBLE REAL	8	-1,7976931348623157 E+308 à 1,7976931348623157 E+308
DECIMAL [(M[,D])]	M+D	
DATE	3	Date au format 'AAAA-MM-JJ' allant de '1000-01-01' à '9999-12-31'
DATETIME	8	Date et heure au format 'AAAA-MM-JJ HH:MM:SS' allant de '1000-01-01 00:00:00' à '9999-12-31 23:59:59'
TIMESTAMP [M]	4	Date sous forme numérique allant de '1970-01-01 00:00:00' à l'année 2037. Selon M, l'affichage varie.
TIME	3	Heure au format 'HH:MM:SS', allant de '-838:59:59' à '838:59:59'
YEAR	1	
BIT	1	Idem CHAR (1)
BOOL	1	Idem CHAR (1)
CHAR (L)	M	Chaîne de L caractères (255 maximum) remplie d'espaces à la fin
VARCHAR (L)	L+1	Chaîne de L caractères (255 maximum) sans espace à la fin
TEXT (L)	L+2	Chaîne de L caractères (65535 maximum)
LONGTEXT (L)	L+4	Chaîne de L caractères (4 294 967 295 maximum)
ENUM ('valeur_possible1', 'valeur_possible2', 'valeur_possible3',...)	1 ou 2 octets	La place occupée est fonction du nombre de solutions possibles : 65 535 valeurs maximum
SET ('valeur_possible1', 'valeur_possible2',...)	1, 2, 3, 4 ou 8 octets	La place occupée est fonction du nombre de solutions possibles : 64 valeurs maximum

Pour SQL Server, se reporter à la documentation.

II. Formats d'échange de données

Les données sont conservées dans un ou des fichiers que **seul** le SGBD gère. Si l'on veut **exporter** les données vers un autre logiciel (SGBD ou non), un format d'échange standard est nécessaire.

1. Le format Texte brut (CSV)

Le plus ancien format, simplissime, ne prend pas en compte les caractères spécifiques nationaux. Il n'est pas adapté à l'échange de plusieurs tables.

2. Le XML

Format récent, le XML est l'inverse du texte brut. Il est évolué, supporte les spécificités des langues, mais demande plus de travail pour le mettre en œuvre. C'est **le format utilisé en priorité** aujourd'hui pour échanger et stocker des données.

Il peut être accompagné d'un fichier *DTD*, qui définit sa règle d'écriture, et éventuellement d'une feuille de style pour améliorer sa lisibilité avec un navigateur.

Exemple de fichier XML :

```
<?xml version="1.0" encoding="UTF-8" ?>
<racine>
  <fruits>
    <kiwis>3</kiwis>
    <mangues>4</mangues>
    <pommes/>
  </fruits>
  <fruits>
    <panier>true</panier>
  </fruits>
  <legumes>
    <patates>bintje</patates>
    <poireaux>>false</poireaux>
  </legumes>
  <viandes>agneau</viandes>
  <viandes>poulet</viandes>
  <viandes>boeuf</viandes>
</racine>
```

3. Le JSON

JSON signifie JavaScript Object Notation. Alternative au XML, il est légèrement plus léger

```
{
  "fruits": [
    { "kiwis": 3,
      "mangues": 4,
      "pommes": null
    },
    { "panier": true }
  ],
  "legumes": {
    "patates": "bintje",
    "poireaux": false
  },
  "viandes": ["agneau", "poulet", "boeuf"]
}
```

III. Aspects sécuritaires sur les formats d'échange

Destinés à transporter des données, les données avec ces formats sont exposés aux altérations de leur contenu.

1. Texte brut/CSV

Aucune protection n'est possible (sauf celle liée au système de fichiers pour le stockage). Extension classique : `.txt`, `.csv`

2. JSON et XML

Ces formats peuvent être **signés électroniquement**.

Un fichier signé atteste :

- Que le fichier provient de la source indiquée (**authentification** de la provenance) ;
- Que le fichier n'a pas été modifié (**intégrité** de son contenu) ;
- Que l'émetteur ne peut nier être à l'origine du document transmis (**non-répudiation**) ;

Un fichier signé n'empêche pas la lecture de son contenu (**non confidentialité**)

Exemple avec XML :

Données à signer :

```
1 <?xml version="1.0" encoding="ISO-8859-1"?>
2 <Envelope xmlns="http://example.org/envelope">
3   <Body>
4     Olá mundo
5   </Body>
6 </Envelope>
```

Document signé :

```
1 <?xml version="1.0" encoding="ISO-8859-1"?>
2 <Envelope xmlns="http://example.org/envelope">
3   <Body>
4     Olá mundo
5   </Body>
6   <Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
7     <SignedInfo>
8       <CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n-20010315" />
9       <SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#rsa-sha1" />
10      <Reference URI="">
11        <Transforms>
12          <Transform Algorithm="http://www.w3.org/2000/09/xmldsig#enveloped-signature" />
13        </Transforms>
14        <DigestMethod Algorithm="http://www.w3.org/2000/09/xmldsig#sha1" />
15        <DigestValue>UWuYTYug10J1k5hKfonxthgrAR8</DigestValue>
16      </Reference>
17    </SignedInfo>
18    <SignatureValue>
19      TSQUoVrQ0kg1eiltNwIhKPrIdsi1VhWjYNJLXvfQqW2EKk3X37X862SCfrz7v8IYJ7OorWw1FpGDStJDSR6sa0
20      ScqSvmesGrGEEq+U6zegR9nH0lvcGZ8Rvc/y7U9kZrE4fHqEiLyfpmzJyPmWUT9Uta14nPJYsl3cmdThHB8Bs=
21    </SignatureValue>
22    <KeyInfo>
23      <KeyValue>
24        <RSAKeyValue>
25          <Modulus>
26            411z0Y3Y9fXoh3Y5f06wBbtTg94Pt6vcfd1KQ0FLm0S36aGJtTSb6pYKfyX7PqCUQ8wgL6xUJ5GRPEsu9gyz8
27            Zobwf2sGCsvu40CWot9fcFB2PfXro1Vtlh/xl/yYHm+Gzqh0Bw76xtLHSfLfpVOrmZdwKmSFKMTvNX0Fd0V18=
28          </Modulus>
29          <Exponent>AQAB</Exponent>
30        </RSAKeyValue>
31      </KeyValue>
32    </KeyInfo>
33  </Signature>
34 </Envelope>
```

Un fichier XML signé peut être vérifié en ligne, par exemple [ici](https://www.aleksey.com/xmlsec/xmlsig-verifier.html) :
<https://www.aleksey.com/xmlsec/xmlsig-verifier.html>